

Blood Cells Classification Project

Technical Documentation

Image Processing & Machine Learning Pipeline

(Ahmed Ashraf , Rawan Hossam , Farida Essam)

Attribute	Details
Project Title	Blood Cells Multi-Class Classification
Platform	Google Colab (GPU-accelerated)
Language	Python 3
Dataset	Kaggle — paultimothymooney/blood-cells (~1.5 GB)
Target Classes	Eosinophil, Lymphocyte, Monocyte, Neutrophil
Image Size	224 × 224 pixels
Total Phases	2 (Image Processing + ML Classification)

1. Project Overview

This project implements a complete pipeline for the automated classification of blood cell types from microscopy images. It is structured in two sequential phases: a classical image-processing phase (Phase 1) and a machine-learning classification phase (Phase 2).

The goal is to distinguish between four white blood cell types — Eosinophils, Lymphocytes, Monocytes, and Neutrophils — using a combination of handcrafted features, deep CNN feature extraction, and classical classifiers.

1.1 Objectives

- Apply classical image processing techniques (contrast enhancement, noise removal, frequency filtering) to blood cell images.
- Extract discriminative features using both handcrafted methods and pre-trained deep CNNs.
- Train and evaluate multiple classifiers (SVM, Random Forest, KNN, XGBoost) on the extracted features.
- Fuse CNN features with handcrafted features to improve classification performance.
- Evaluate models using robust metrics: Accuracy, F1-Score, AUC-ROC, Cohen's Kappa, Sensitivity, and Specificity.

1.2 Dataset

The dataset used is the publicly available Blood Cell Images dataset from Kaggle (paultimothymooney/blood-cells), which contains JPEG microscopy images organized into four class folders.

Class	Cell Type	Role in Immune System
EOSINOPHIL	Granulocyte	Fights parasites and allergic reactions
LYMPHOCYTE	Agranulocyte	Adaptive immune response (T/B cells)
MONOCYTE	Agranulocyte	Phagocytosis; becomes macrophage in tissue
NEUTROPHIL	Granulocyte	First responder to bacterial infection

2. Phase 1 — Classical Image Processing

Phase 1 establishes the image preprocessing pipeline. It operates on raw grayscale images and applies a sequential series of enhancements and filters before saving outputs for inspection.

2.1 Libraries & Setup

- `os`, `numpy` — file system operations and numerical arrays
- `PIL` (Pillow) — image loading and saving
- `matplotlib` — visualization and histogram plotting
- Google Colab drive — cloud storage mount for dataset access

Dataset and output paths are configurable constants:

```
dataset_path = "/content/drive/MyDrive/BloodCells_Project"
output_path  = "/content/drive/MyDrive/BloodCells_Output"
```

2.2 Core Utility Functions

2.2.1 `load_image_grayscale(path)`

Loads a single image from disk, converts it to 8-bit grayscale using PIL's 'L' mode, and returns it as a NumPy uint8 array. All downstream processing in Phase 1 operates on this format.

2.2.2 `contrast_stretch(image)`

Performs linear contrast stretching (normalization) to use the full [0, 255] intensity range. The formula maps the minimum pixel value to 0 and the maximum to 255, enhancing visual distinction between cell regions and background.

```
stretched = (image - r_min) * (255.0 / (r_max - r_min))
```

A guard condition handles constant images (`r_min == r_max`) to avoid division by zero.

2.2.3 `manual_histogram_equalization(image)`

Implements histogram equalization from scratch without library helpers. The function:

- Computes the pixel-value histogram manually (256 bins).
- Calculates the Probability Density Function (PDF) by normalizing the histogram.
- Derives the Cumulative Distribution Function (CDF) iteratively.
- Creates a lookup table mapping old pixel values to equalized ones via: `transform = round(255 × CDF)`.
- Applies the mapping pixel-by-pixel to produce the equalized output.

Returns the equalized image, the raw histogram, and the CDF — all used for visualization.

2.2.4 `add_salt_pepper_noise(image, salt_prob=0.01, pepper_prob=0.01)`

Simulates salt-and-pepper noise by randomly setting pixels to 255 (salt/white) or 0 (pepper/black) according to the given probabilities. Used to create a noisy version of the equalized image for denoising demonstration.

2.2.5 manual_median_filter(image, kernel_size=3)

A fully manual spatial median filter. The image is edge-padded, then for every pixel, the filter extracts a `kernel_size × kernel_size` neighborhood, sorts the values, and replaces the center pixel with the median. This effectively removes salt-and-pepper noise while preserving edges.

2.2.6 frequency_low_pass_filter(image, radius=30)

Applies a circular ideal low-pass filter in the frequency domain:

- Computes the 2D FFT and shifts the zero-frequency component to the center.
- Creates a binary circular mask (1 inside the radius, 0 outside).
- Multiplies the shifted spectrum by the mask, removing high-frequency components.
- Inverse FFT is applied, and the result is normalized to `[0, 255]`.

Result: a smoothed image retaining low-frequency (shape/global) information.

2.2.7 frequency_high_pass_filter(image, radius=30)

Inverse of the low-pass filter: the mask is 0 inside the radius and 1 outside, preserving only high-frequency components (edges, fine texture). Steps are otherwise identical.

2.3 Main Processing Pipeline — process_image()

The central function orchestrates all processing steps for a single image:

Step	Operation	Output File Suffix
1	Load original image (grayscale)	_1_original.png
2	Contrast stretching	_2_contrast_stretched.png
3	Histogram equalization	_3_hist_equalized.png
4	Add salt-and-pepper noise (demo)	_4_noisy.png
5	Median filter denoising	_5_median_filtered.png
6	Frequency filtering (low or high)	_6_frequency_filtered.png
7	Save frequency mask	_7_frequency_mask.png

All steps are also displayed in a 2×3 subplot grid and histograms of the original and equalized images are plotted. The function accepts a `freq_mode` parameter ('low' or 'high') to switch the frequency filter type.

2.4 Batch Processing

Phase 1 concludes by scanning the dataset directory for all supported image types (`.png`, `.jpg`, `.jpeg`, `.bmp`, `.tif`, `.tiff`), then processing up to 5 images using the full pipeline with low-pass filtering enabled.

3. Phase 2 — Machine Learning Classification

Phase 2 builds on the dataset to train and evaluate a set of classifiers. It covers data loading, augmentation, handcrafted feature extraction, deep CNN feature extraction, feature fusion, dimensionality reduction, cross-validation, and visualization.

3.1 Environment & Configuration

- TensorFlow / Keras — GPU detection and deep learning backbone.
- scikit-learn — preprocessing, classifiers, metrics, cross-validation.
- OpenCV (cv2) — image I/O, color space conversion, ORB feature detection.
- scikit-image — LBP and GLCM texture feature extraction.
- XGBoost — gradient-boosted tree classifier.
- Kaggle API — automated dataset download (~1.5 GB).

Key global constants:

Constant	Value	Purpose
IMG_SIZE	224	Resize all images to 224×224 pixels
BATCH_SIZE	32	Mini-batch size for CNN inference
CLASSES	4 classes	EOSINOPHIL, LYMPHOCYTE, MONOCYTE, NEUTROPHIL
RANDOM_STATE	42	Global seed for reproducibility
PCA_COMPONENTS	100	Dimensionality after PCA reduction
CV_FOLDS	5	Number of folds for cross-validation
OUTPUT_DIR	/content/outputs	Directory for all saved plots and figures

3.2 Data Loading & Augmentation

3.2.1 load_images()

Iterates through each class folder, reads all .jpg/.jpeg/.png images using OpenCV, converts from BGR to RGB, resizes to 224×224, and appends to a list. Returns stacked NumPy arrays for images (uint8) and string labels.

3.2.2 augment_images(images, labels, target=700)

Balances the dataset to a target of 700 samples per class using Keras' ImageDataGenerator with the following augmentations:

- Random rotation up to ± 15 degrees
- Horizontal flipping
- Random zoom up to 10%

- Width and height shifts up to 5%

Only underrepresented classes are augmented; classes already at or above the target are left unchanged.

3.3 Feature Extraction

Two feature extraction strategies are employed, and their outputs are ultimately fused:

3.3.1 Handcrafted Features (162 dimensions total)

Feature Type	Function	Dimensions	Description
LBP	feat_lbp()	10	Local Binary Pattern — encodes local texture using a uniform pattern histogram (radius=1, 8 neighbors).
GLCM	feat_glcm()	24	Gray-Level Co-occurrence Matrix — computes contrast, dissimilarity, homogeneity, energy, correlation, and ASM across 4 angles.
Color Histogram	feat_color()	96	Per-channel (R, G, B) histogram with 32 bins each; captures color distribution.
ORB Descriptor	feat_orb()	32	Oriented FAST and Rotated BRIEF — detects up to 50 keypoints, averages their 32-dim descriptors.

Feature importance is visualized using a Random Forest trained on the 162-dimensional space. The importance bar chart is color-coded by feature group (LBP, GLCM, Color, ORB).

3.3.2 CNN Deep Features

Three pre-trained ImageNet models are used as feature extractors (include_top=False, pooling='avg'). Each produces a global average pooled feature vector:

Architecture	Output Dim	Preprocessing
VGG16	512	vgg16.preprocess_input
ResNet50	2048	resnet50.preprocess_input
EfficientNetB0	1280	efficientnet.preprocess_input

Inference is performed in mini-batches to avoid memory overflows. ResNet50 features are cached separately for later use in feature fusion.

3.4 Dimensionality Reduction — PCA

After extraction, all feature sets are standardized (zero mean, unit variance via StandardScaler) and reduced using PCA to 100 components. The reduce() function adapts the number of components so it never exceeds the number of training samples minus 1 or the number of features, preventing numerical errors.

PCA explained-variance plots and per-component weight breakdowns by feature group (LBP / GLCM / Color / ORB / ResNet50) are saved as figures.

3.5 Feature Fusion

The ResNet50 CNN features (2048-dim) and all handcrafted features (162-dim) are horizontally concatenated to form a fused feature vector of 2210 dimensions. This is then reduced via PCA to 100 components.

```
Fused = np.hstack([ResNet50_features, Handcrafted_features]) # shape: (N, 2210)
```

Fusion is hypothesized to combine the global representational power of deep features with the domain-specific discriminability of handcrafted features.

3.6 Classifiers

Four classifiers are evaluated on every feature set:

Classifier	Key Hyperparameters	Notes
SVM	kernel=rbf, C=10, probability=True	RBF kernel; probability calibration enabled for AUC computation.
Random Forest	n_estimators=200	Ensemble of 200 decision trees with bootstrapping.
KNN	n_neighbors=7	7-nearest neighbors in PCA-reduced space.
XGBoost	n_estimators=200, max_depth=6, lr=0.1	Gradient-boosted trees with multi-class log-loss.

3.7 Evaluation Metrics

Each model is evaluated on the held-out test set (20% split). The evaluate() function computes:

Metric	Definition
Accuracy	Overall fraction of correctly classified samples.
Sensitivity (Recall)	Macro-averaged true positive rate across all classes.
Specificity	Macro-averaged true negative rate per class (computed from confusion matrix).
Precision	Macro-averaged positive predictive value.
F1-Score	Harmonic mean of Precision and Recall (macro-averaged).
AUC-ROC	Area Under the ROC Curve using One-vs-Rest macro-averaging.
Cohen's Kappa	Agreement beyond chance; robust to class imbalance.

3.8 Cross-Validation

The best pipeline (SVM on ResNet50+Handcrafted fused features) is further validated using Stratified 5-Fold Cross-Validation to ensure performance stability. Each fold independently applies StandardScaler and PCA to prevent data leakage from the test fold.

Results are plotted as a per-fold Accuracy and F1-Score line graph with mean-value horizontal reference lines.

3.9 Visualization Outputs

Output File	Description
class_distribution.png	Bar chart of sample counts per class after augmentation.
sample_images.png	4×4 grid of sample images (one row per class).
handcrafted_feature_importance.png	Top-20 handcrafted features ranked by Random Forest Gini importance.
pca_variance.png	Cumulative explained variance curve for fused PCA (with 95% threshold line).
pca_component_weights.png	Mean absolute PCA component weights broken down by feature group.
rf_feature_importance.png	Top-20 PCA components by Random Forest importance on fused features.
xgb_feature_importance.png	Top-20 PCA components by XGBoost F-score importance on fused features.
cross_validation.png	5-fold CV Accuracy and F1-Score plot for the best SVM pipeline.
Confusion_Matrix_SVM_Fusion.png	Confusion matrix for the best model (SVM on fused features).
ROC_Curves_SVM_Fusion.png	Per-class ROC curves with AUC scores for the best model.
model_comparison.png	Side-by-side bar chart comparing Accuracy and F1 for all model-feature combinations.

4. Full Pipeline Summary

Stage	Inputs	Operations	Outputs
Data Ingestion	Raw JPEG images (4 classes)	Load → RGB convert → Resize 224×224 → Augment to 700/class	Balanced image array (N × 224 × 224 × 3)
Phase 1 Processing	Grayscale images	Contrast stretch → Hist. EQ → Add noise → Median filter → FFT filter	7 processed images per input + histograms
CNN Feature Extraction	RGB images (uint8)	VGG16 / ResNet50 / EfficientNetB0 (global avg pooling)	512 / 2048 / 1280-dim feature vectors
Handcrafted Extraction	RGB images	LBP + GLCM + Color Hist + ORB	162-dim feature vectors
Feature Fusion	ResNet50 + HC features	Horizontal concatenation	2210-dim fused vectors
Dimensionality Reduction	Any feature set	StandardScaler + PCA (100 components)	100-dim normalized vectors
Classification	PCA-reduced features	SVM / RF / KNN / XGBoost training + predict	Per-model accuracy, F1, AUC, Kappa
Evaluation & Visualization	Predictions + probabilities	Confusion matrix, ROC curves, model comparison, 5-fold CV	Saved figures + results table

5. Known Issues & Notes

5.1 Bug — Syntax Error in `process_image()`

In the Phase 1 `process_image()` function, the following line contains a stray character 'c' after the string literal, which will cause a `SyntaxError` at runtime:

```
freq_title = "Low-Pass Filtered"c ← invalid syntax
```

Fix: Remove the trailing 'c':

```
freq_title = "Low-Pass Filtered"
```

5.2 Manual Median Filter Performance

The `manual_median_filter()` implementation uses nested Python for-loops over every pixel. For 224×224 images this is extremely slow compared to `scipy.ndimage.median_filter` or `cv2.medianBlur`. This is acceptable for educational demonstration but is not suitable for production use or large-scale batch processing.

5.3 Manual Histogram Equalization Performance

Similarly, `manual_histogram_equalization()` uses a pixel-by-pixel loop for the mapping step. For large images this is orders of magnitude slower than `cv2.equalizeHist()`. Consider vectorizing with NumPy indexing: `equalized = transform[image]`.

5.4 Feature Extraction Cell Re-declaration

The Feature Extraction cell in Phase 2 re-declares all helper functions (`load_images`, `augment_images`, etc.) and re-runs the full data-loading pipeline to ensure the cell is independently executable. This creates redundancy but improves notebook robustness in Colab's stateless execution environment.

5.5 ORB Color Space

The `feat_orb()` function converts using `cv2.COLOR_BGR2GRAY` even though images in the pipeline are stored as RGB. This is a minor inconsistency; because both color orders yield different grayscale values, it could marginally affect keypoint detection quality.

6. Dependencies

Library	Version (recommended)	Purpose
numpy	>=1.21	Array operations, FFT, random noise
Pillow (PIL)	>=8.0	Image I/O for Phase 1
matplotlib	>=3.4	All visualization and histogram plotting
opencv-python (cv2)	>=4.5	Image loading, resizing, ORB, color conversion
scikit-image	>=0.18	LBP and GLCM feature extraction
scikit-learn	>=0.24	ML classifiers, PCA, metrics, CV
tensorflow / keras	>=2.6	CNN backbone models (VGG16, ResNet50, EfficientNetB0)
xgboost	>=1.4	XGBoost classifier
kaggle	any	Dataset download via Kaggle API
pandas	>=1.3	Results table display (Colab display())

End of Documentation